

点亮LED灯

点亮LED灯

- 一、学习目标
- 二、硬件搭建
- 三、实验步骤
 - 1. 打开SYSCONFIG配置工具
 - 2. 参数设置
 - 2.1 设置端口参数
 - 2.2 设置引脚参数
 - 2.3 保存更新配置文件
 - 3. 写入程序
 - 4. 编译
- 四、程序分析
- 五、实验现象

一、学习目标

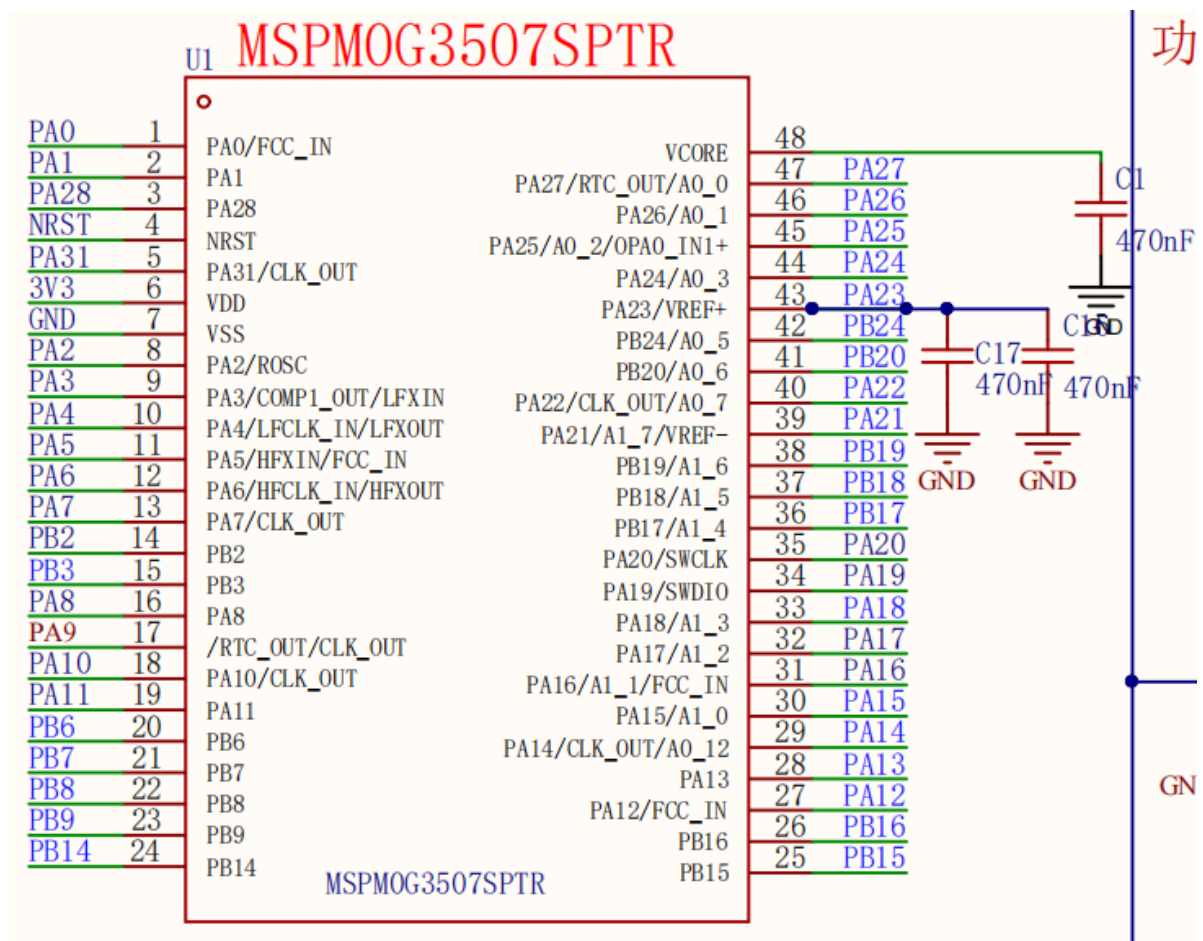
- 1.学习MSPM0G3507主板的引脚基本使用。
- 2.了解如何控制板载的LED灯。

二、硬件搭建

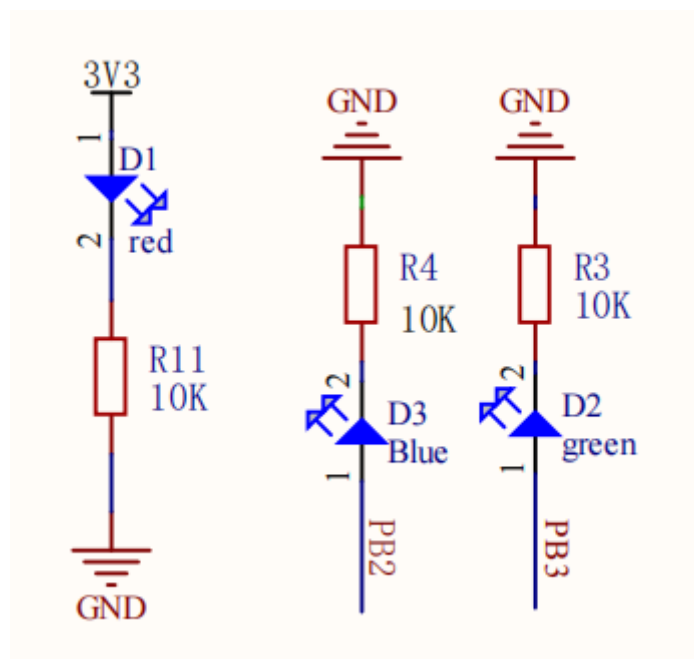
本次课程无需额外的硬件设备，直接利用MSPM0G3507主板上的板载LED灯即可。

我们在主板上设置了两个LED用户灯，用户可以自行DIY其功能。下面以**LED1**例。

MSPM0G3507主控图



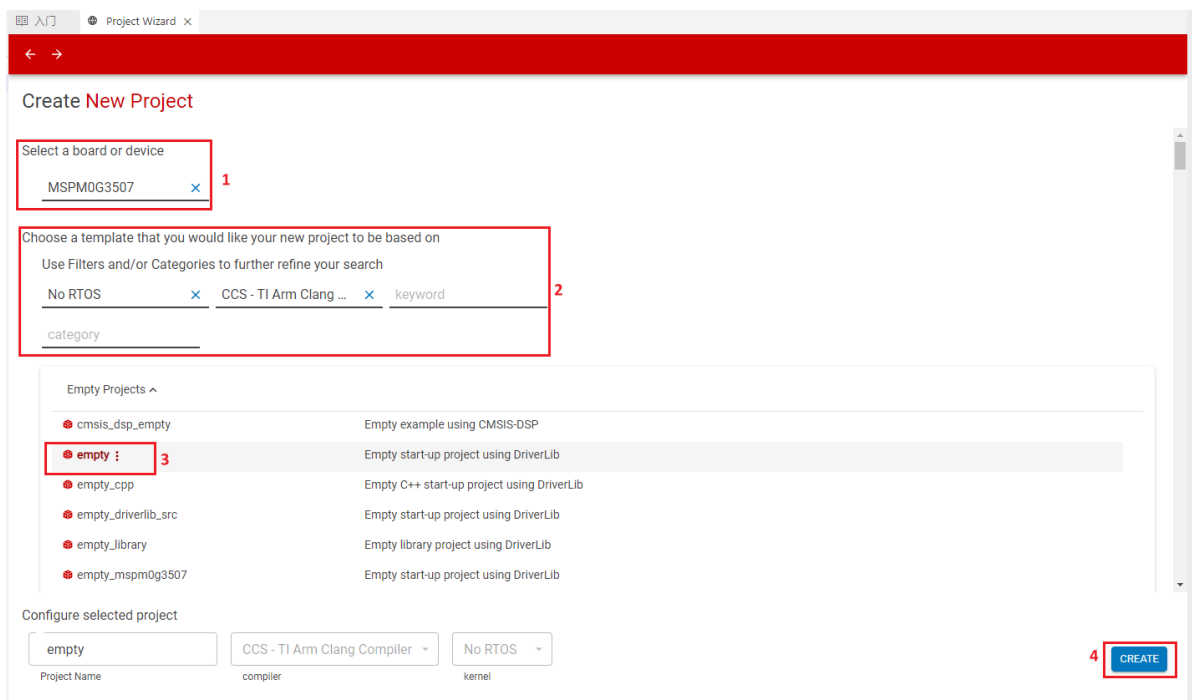
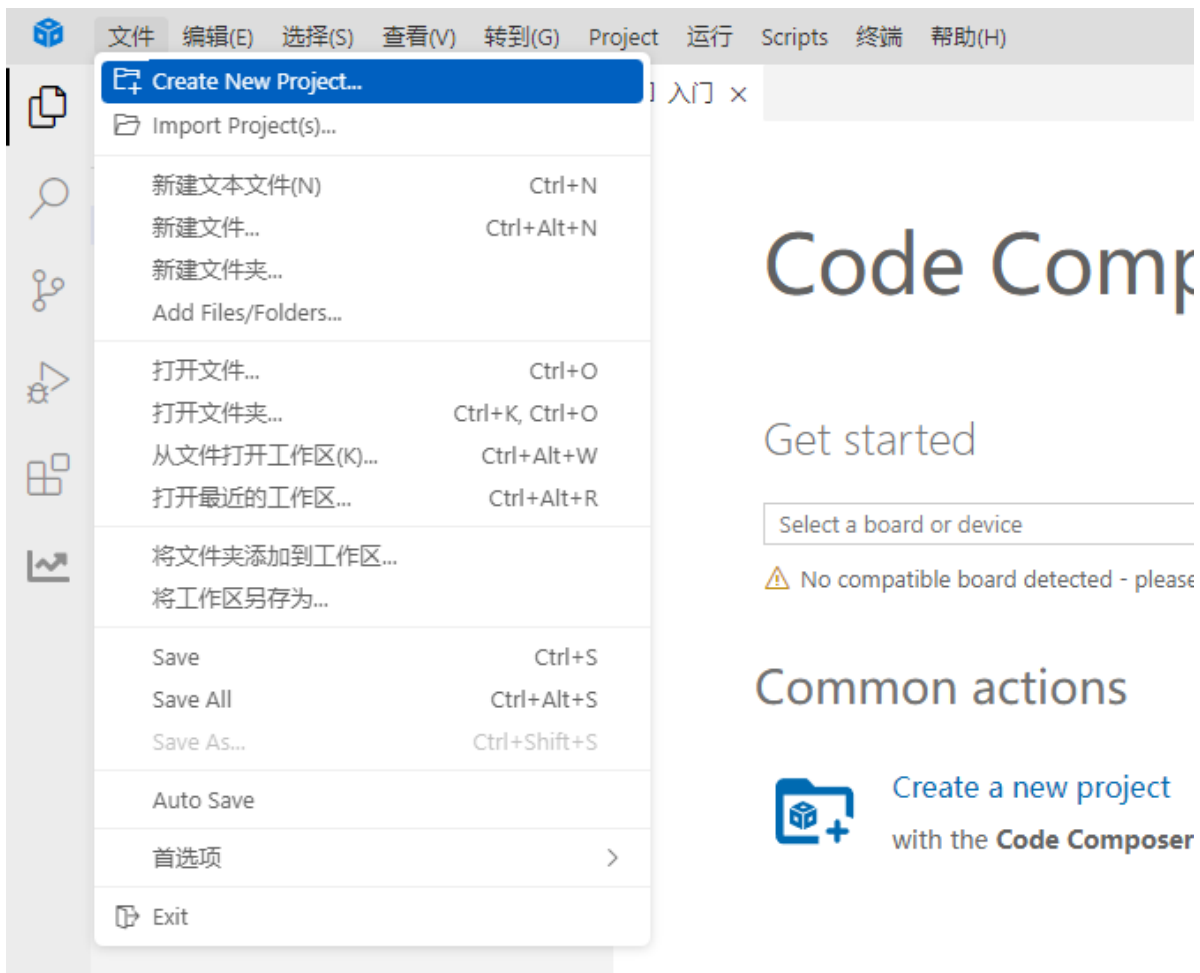
LED部分原理图



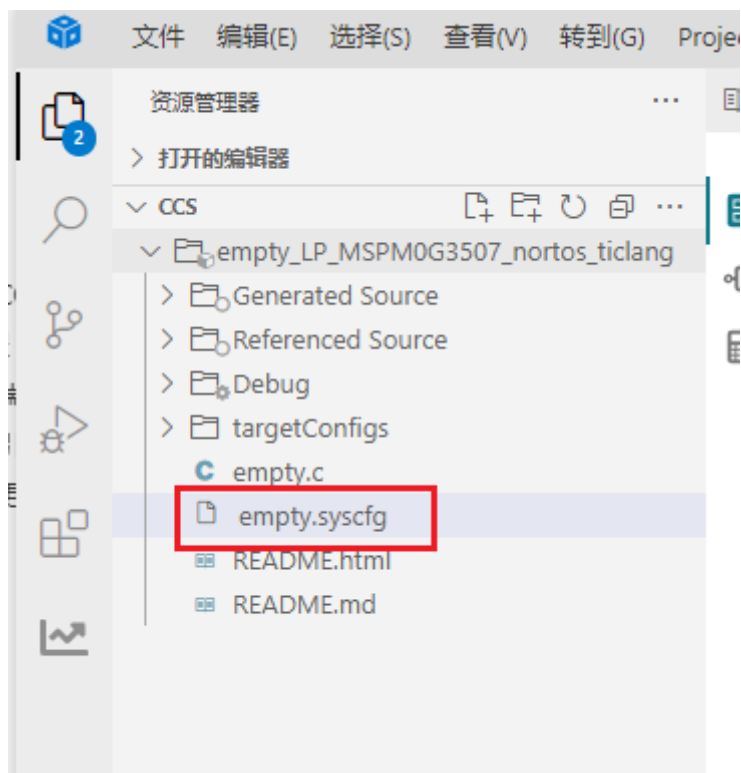
三、实验步骤

1. 打开SYSCONFIG配置工具

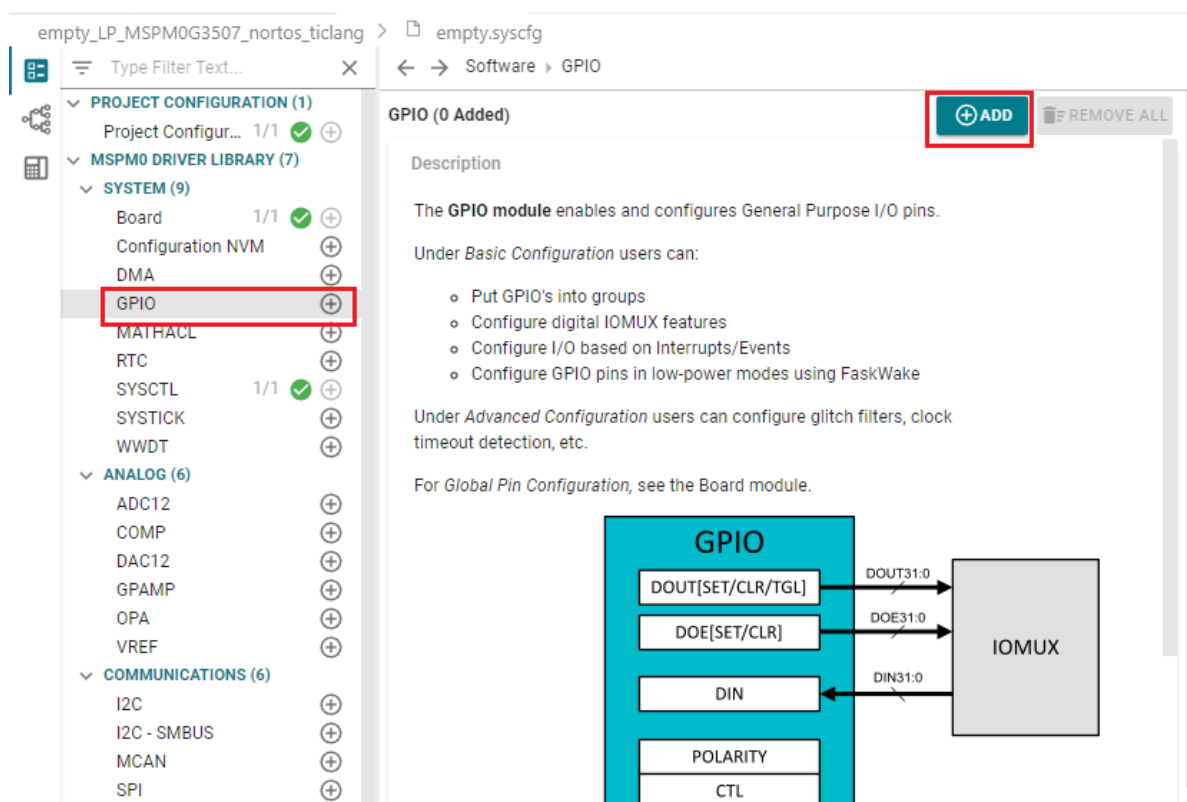
在CCS中新建一个空白工程 empty。



在CCS的左侧工作区中找到并打开empty.syscfg文件。



在 SYSCONFIG 中，左侧选择 MCU 外设，找到 GPIO 选项并点击进入。在 GPIO 中点击 ADD，添加一组GPIO。



2. 参数设置

2.1 设置端口参数

设置GPIO参数，将该引脚命名为 LED1

由LED部分原理图可知，LED1所在的端口为 PB2，所以设置Port为 PORTB。

Name: 用户自定义的引脚名称，设置为PIN_2。

Direction: 设置引脚模式，输入和输出。这里我们是控制LED灯，选择为输出模式。

Initial Value: 设置引脚的初始状态，只有在配置为输出模式时才可以设置。有两个选项，清除和置位。清除即输出低电平，置位即输出高电平。

IO Structure: 设置IO结构。有多个选项，默认(Any)、标准(Standard)、唤醒(Standard with Wake)、高速(High-Speed)以及可以容忍5V(5V Tolerant Open Drain)的结构。

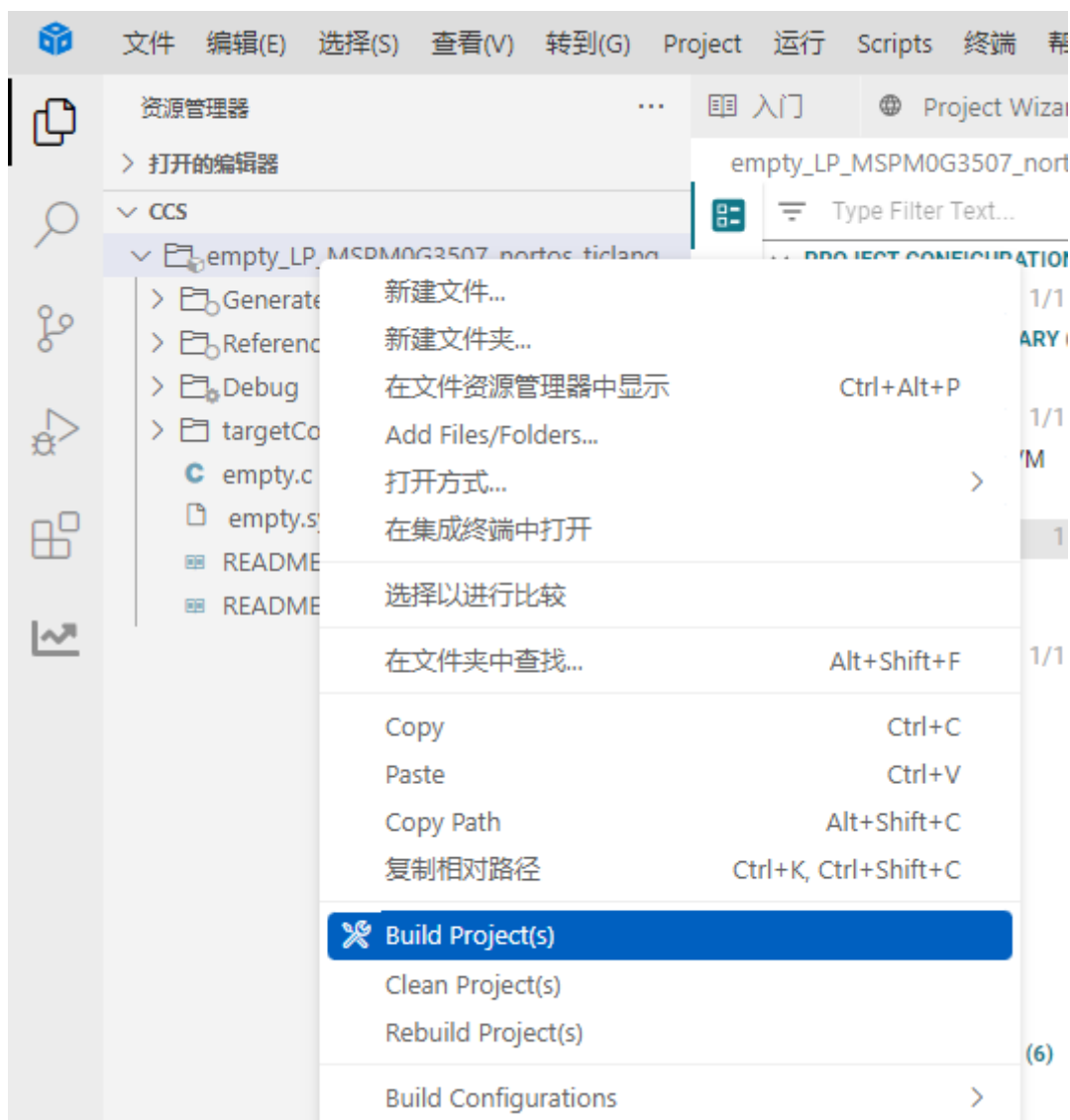
Internal Resistor: 设置引脚的上下拉电阻。有三个选项，不设置、设置上拉和设置下拉电阻。这里根据LED灯的接法，选择了下拉电阻。

Assigned Pin: 设置引脚号。要控制哪一个引脚就填写对应的引脚号，比如开发板中的LED是接到了GPIOB2，那么就填入2。

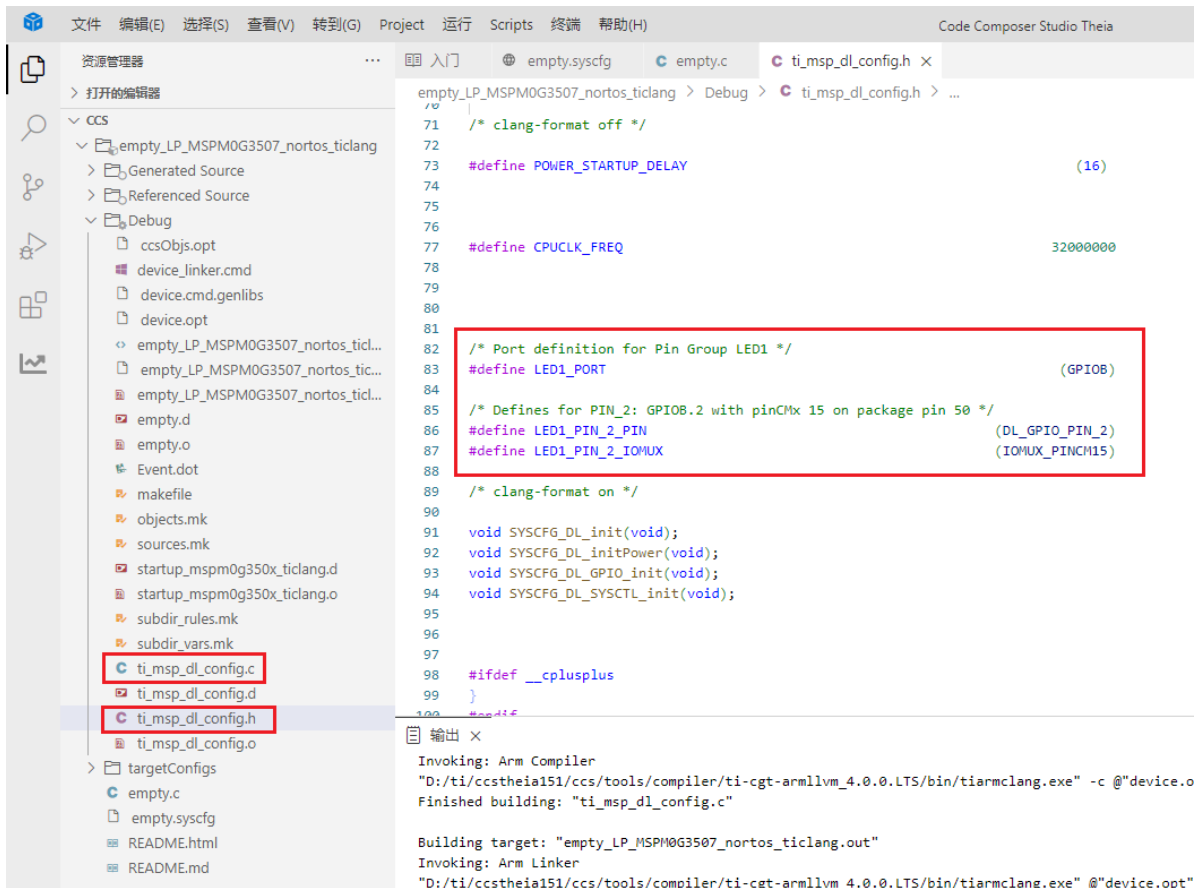
2.3 保存更新配置文件

通过快捷键 Ctrl+S 将.syscfg文件中的配置保存。

保存后编译



编译之后，我们可以看到在Debug文件夹下多了两个文件，`ti_msp_d1_config.c` 和 `ti_msp_d1_config.h`。这两个文件就是基于刚才我们保存的.syscfg文件生成的外设底层驱动配置代码。



3. 写入程序

在empty.c文件中，写入以下代码

```
#include "ti_msp_dl_config.h"

//自定义延时（不精确） Custom delay (not precise)
void delay_ms(unsigned int ms)
{
    unsigned int i, j;
    // 下面的嵌套循环的次数是根据主控频率和编译器生成的指令周期大致计算出来的，
    // 需要通过实际测试调整来达到所需的延时。
    // The number of nested loops below is roughly calculated based on the master
    control frequency and the instruction cycle generated by the compiler,and needs
    to be adjusted through actual testing to achieve the required delay.
    for (i = 0; i < ms; i++)
    {
        for (j = 0; j < 8000; j++)
        {
            // 仅执行一个足够简单以致于可以预测其执行时间的操作
            // Perform only one operation that is simple enough to predict its
            execution time
            __asm__("nop"); // "nop" 代表“无操作”，在大多数架构中，这会消耗一个或几个时钟
            周期 "nop" stands for "no operation", which on most architectures consumes one or
            a few clock cycles
        }
    }
}

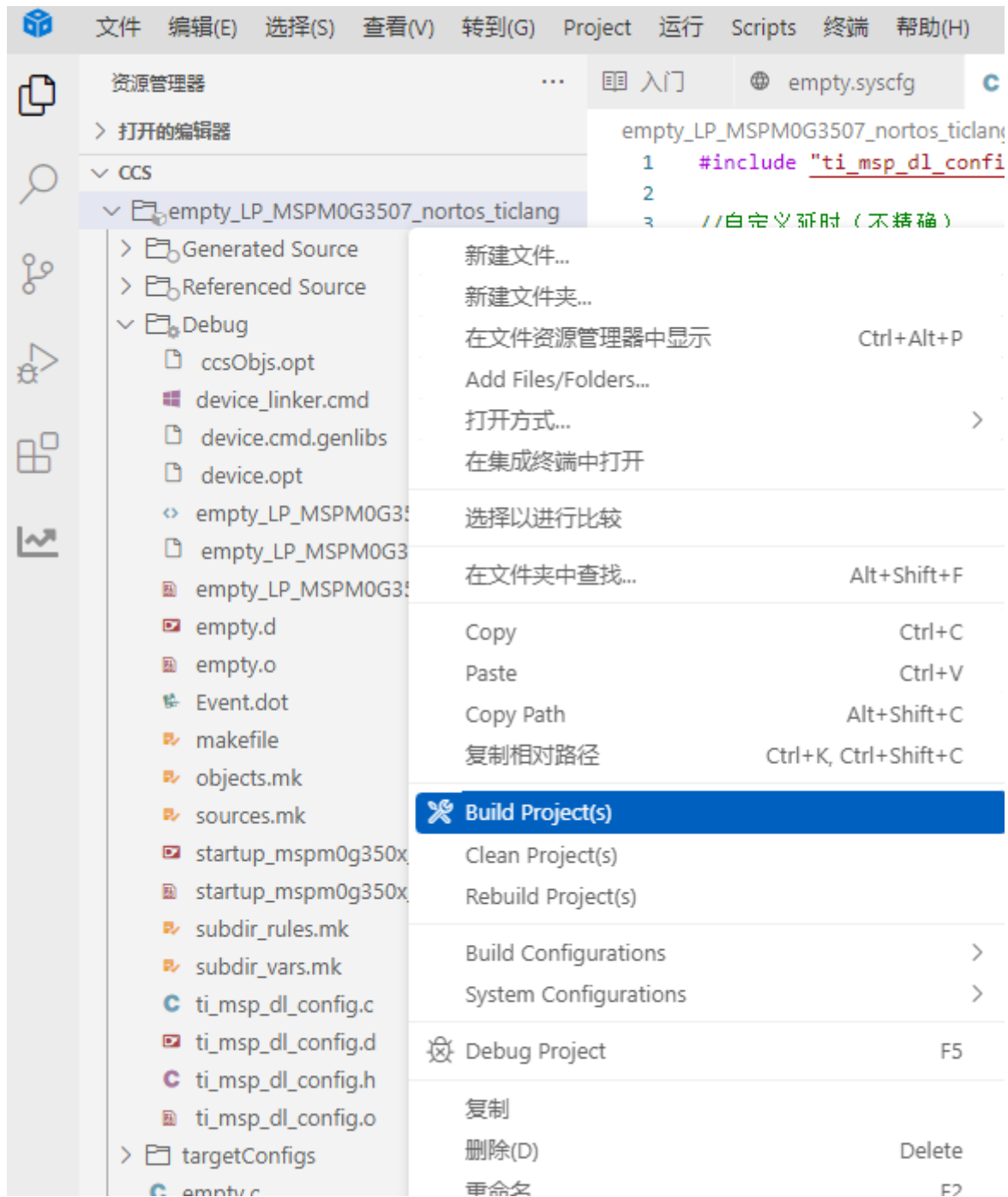
int main(void)
{
    SYSCFG_DL_init();
}
```

```

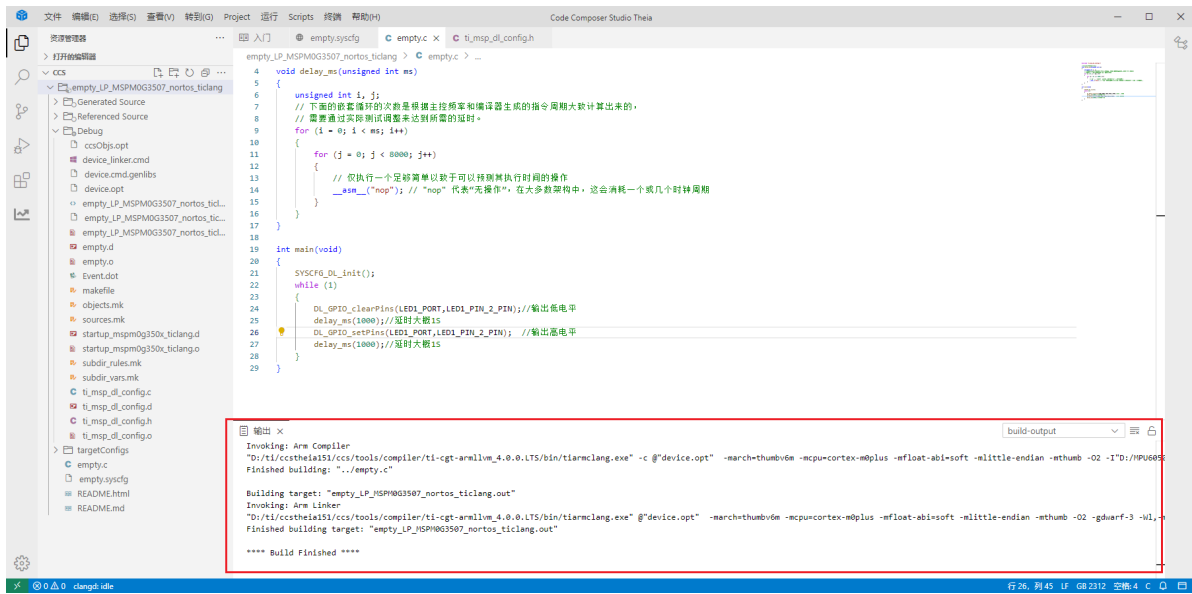
while (1)
{
    DL_GPIO_clearPins(LED1_PORT,LED1_PIN_2_PIN); //输出低电平 Output low level
    delay_ms(1000); //延时大概1S Delay about 1S
    DL_GPIO_setPins(LED1_PORT,LED1_PIN_2_PIN); //输出高电平 Output high level
    delay_ms(1000); //延时大概1S Delay about 1S
}
}

```

4. 编译



我这里显示编译成功了，即可将程序下载到开发板。



四、程序分析

- dl_gpio.h



__STATIC_INLINE void DL_GPIO_setPins(GPIO_Regs* gpio, uint32_t pins): 该函数为控制引脚输出高电平。

__STATIC_INLINE void DL_GPIO_clearPins(GPIO_Regs* gpio, uint32_t pins): 该函数为控制引脚输出低电平。

- empty.c

```

empty_LP_MSPM0G3507_nortos_ticlang > C empty.c > ...
1  #include "ti_msp_dl_config.h"
2
3  //自定义延时 (不精确) Custom delay (not precise)
4  void delay_ms(unsigned int ms)
5  {
6      unsigned int i, j;
7      // 下面的嵌套循环的次数是根据主控频率和编译器生成的指令周期大致计算出来的,
8      // 需要通过实际测试调整来达到所需的延时。
9      // The number of nested loops below is roughly calculated based on the master control frequency and the instruction cycle gener
10     for (i = 0; i < ms; i++)
11     {
12         for (j = 0; j < 8000; j++)
13         {
14             // 仅执行一个足够简单以致于可以预测其执行时间的操作
15             // Perform only one operation that is simple enough to predict its execution time
16             __asm__("nop"); // "nop" 代表"无操作", 在大多数架构中, 这会消耗一个或几个时钟周期 "nop" stands for "no operation", which on
17         }
18     }
19 }
20
21 int main(void)
22 {
23     SYSCFG_DL_init();
24     while (1)
25     {
26         DL_GPIO_clearPins(LED1_PORT, LED1_PIN_2_PIN); //输出低电平 Output low level
27         delay_ms(1000); //延时大概1s Delay about 1S
28         DL_GPIO_setPins(LED1_PORT, LED1_PIN_2_PIN); //输出高电平 Output high level
29         delay_ms(1000); //延时大概1s Delay about 1S
30     }
31 }

```

采用了不精确延时，设置PB2输出低电平，延时大概1s后输出高电平，再延时大概1s。循环此进程。

五、实验现象

程序下载完成之后，可以看到MSPM0开发板上的LED1灯被点亮，并每隔大约1s闪烁一次。

